The Knot: A Verified Self-Reader for the Two-Move Calculus

Working paper v0.1, Project secret3, August 11, 2026. Movement four, closing the C5 campaign.

Abstract

This paper delivers the summit of the reader campaign: simT, a closed term of the two-move calculus that reads quoted programs, and the pair of theorems establishing that it agrees with the reference engine on every code. The construction is three definitions on the standing bench: two stitching helpers that rebuild a just around a repaired child, a descent body that asks itself about the left child and then the right, and the step function simF whose root branch is movement three's dispatcher. The knot itself is self-application: simT is zSim applied to zSim, where zSim abstracts one variable and applies simF to that variable's self-application, so that a single vanishing-engine beta is the whole unfolding lemma. Correctness is one mutual induction over codes, mirroring the engine's nine clauses, with the unfolding entering every chain through one shared lemma and the movement-two congruence pattern carrying the recursive calls in argument position:

```
simCorrectJ : simStep c = Just c' ->
              Steps (App simT (quote c)) (jTag (quote c'))
simCorrectN : simStep c = Nothing ->
              Steps (App simT (quote c)) nTag
```

The whole of part X compiled on the first attempt, the fifth consecutive first-attempt compilation of the campaign, which is reported not as luck but as the measured value of the lemma catalog. Corollaries in prose close the paper, including the one the campaign was named for: the reader, being itself a closed term, falls under its own theorems.

1. The construction

Three terms, no new bench machinery. The stitchers ljT and rjT take the untouched sibling and the repaired child and answer just of the constructor's rebuild; each is a two-argument abstraction whose uses are single abs2Beta instances. The descent dTerm receives self and the two children: it asks self about the left child and cases the tag; a just is stitched left; a nothing forwards to the right child, whose just is stitched right and whose nothing is final. The step function simF cases its quoted input through the dispatcher: a firing root is rewrapped by jWrap; a silent root enters the descent with self threaded through. The knot is the classical self-application, and because the seed zSim has a concrete body, its unfolding is one absCorrect: simUnfold says simT walks to simF applied to simT, and the definitional splice does the rest, the checker evaluating the substitution into the seed's concrete tree.

2. The induction

The correctness proof mirrors the engine clause for clause, nine cases each way, and three mechanization findings made it routine. First, as-patterns: the engine descends into the left child as a whole, so the induction must recurse on that child as a whole, and binding it with an as-pattern keeps the recursion visibly structural. Second, the with-proof idiom dispatches the engine's internal case analyses, with the recorded equation feeding the inductive hypothesis directly; the impossible branches of the silent theorem discharge themselves. Third, the descent bridges descRJust

and descRNothing, proved back in stage two, translate the engine's right-descent wrapper into exactly the witnesses the chains need; nothing about the engine had to be reproved. Every chain is then catalog work: the shared entry lemma, the dispatcher theorem for the shape, one tag case, the shared descent entry, congruence-lifted inductive hypotheses, and a stitcher. The longest clause is eleven links.

3. What now stands, counted

The campaign, complete: a quoted alphabet with closure and injectivity; a total reference engine with soundness, silence, and run theorems; a bench of substitution lemmas through arity four whose every side condition is discharged by one closure theorem; internal tags, an internal constructor, two verified eyes, a verified dispatcher, and a verified reader tied by its own knot, agreeing with the engine on all codes. Sixty two named theorems across the campaign's six papers, in a module of 3,337 lines, total, without postulates or holes.

4. Corollaries in prose

Three sentences the compiled pieces now jointly authorize. With the engine's soundness: every just the reader returns names a real step of the calculus, so the reader's answers are never merely formal. With strong normalization: pointed at the quote of any typed program and iterated, the reader's own answers bottom out, the reading always finishes. And the campaign's namesake: simT is a closed term of the calculus it reads, so there is a code whose embedding is simT itself, and the correctness theorems, quantified over every code, cover that one too; the machine that runs itself, exhibited, reads itself, and the checker has signed the whole sentence. The book's Part One closes on precisely this receipt.

Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 3,337 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification.

Appendix B: The knot section (module part X)

```
----------------------------------------------------------------------
-- Tier 2, part X: the knot (C5 stage three, movement four)
----------------------------------------------------------------------

||| helpers rebuilding a just around a stitched application
ljT : Term
ljT = abs "rr" (abs "xp"
  (Proofs.jTag (App (App Proofs.qMk (Var "xp")) (Var "rr"))))

rjT : Term
rjT = abs "ll" (abs "yp"
  (Proofs.jTag (App (App Proofs.qMk (Var "ll")) (Var "yp"))))

||| the descent body: ask self about the left child; on just,
||| stitch left; on nothing, ask about the right; on just, stitch
||| right; on nothing, answer nothing
dTerm : Term
dTerm = abs "sf" (abs "l" (abs "r"
  (App (App (App (Var "sf") (Var "l"))
             (App (App (App (Var "sf") (Var "r")) Proofs.nTag)
                  (App Proofs.rjT (Var "l"))))
       (App Proofs.ljT (Var "r")))))
```

```
||| the step function: the dispatcher at the root, the descent below
simF : Term
simF = abs "s" (abs "q"
  (App (App (App Proofs.rootRead (Var "q"))
             (App (App (App (Var "q") Proofs.nTag) Proofs.nTag)
                  (App Proofs.dTerm (Var "s"))))
       Proofs.jWrap))

||| the knot: the reader is the self-application of its own seed
zSim : Term
zSim = abs "x" (App Proofs.simF (App (Var "x") (Var "x")))

simT : Term
simT = App Proofs.zSim Proofs.zSim

||| one unfolding of the knot
simUnfold : Steps Proofs.simT (App Proofs.simF Proofs.simT)
simUnfold =
  absCorrect "x" (App Proofs.simF (App (Var "x") (Var "x"))) Proofs.zSim

||| entering the body on a quoted input
simEnter : (c : Code) ->
  Steps (App Proofs.simT (Proofs.quote c))
        (App (App (App Proofs.rootRead (Proofs.quote c))
                   (App (App (App (Proofs.quote c) Proofs.nTag)
                             Proofs.nTag)
                        (App Proofs.dTerm Proofs.simT)))
             Proofs.jWrap)
simEnter c =
  stepsTrans (appLs simUnfold) $
  abs2Beta "s" "q"
    (App (App (App Proofs.rootRead (Var "q"))
               (App (App (App (Var "q") Proofs.nTag) Proofs.nTag)
                    (App Proofs.dTerm (Var "s"))))
         Proofs.jWrap)
    Proofs.simT (quote c) (quoteClosed "s" c)

||| entering the descent on an application
dEnter : (x, y : Code) ->
  Steps (App (App (App (Proofs.quote (QApp x y)) Proofs.nTag)
             Proofs.nTag)
        (App Proofs.dTerm Proofs.simT))
        (App (App (App Proofs.simT (Proofs.quote x))
                   (App (App (App Proofs.simT (Proofs.quote y))
                             Proofs.nTag)
                        (App Proofs.rjT (Proofs.quote x))))
             (App Proofs.ljT (Proofs.quote y)))
dEnter x y =
  stepsTrans (qAppCase x y nTag nTag (App dTerm simT)) $
  abs3Beta "sf" "l" "r"
    (App (App (App (Var "sf") (Var "l"))
               (App (App (App (Var "sf") (Var "r")) Proofs.nTag)
                    (App Proofs.rjT (Var "l"))))
         (App Proofs.ljT (Var "r")))
```

```
    Proofs.simT (quote x) (quote y)
    (quoteClosed "sf" x) (quoteClosed "sf" y) (quoteClosed "l" y)

||| stitching a left answer
dLeftJ : (x', y : Code) -> (right : Term) ->
  Steps (App (App (Proofs.jTag (Proofs.quote x')) right)
             (App Proofs.ljT (Proofs.quote y)))
        (Proofs.jTag (Proofs.quote (QApp x' y)))
dLeftJ x' y right =
  stepsTrans (jTagCase (quote x') right (App ljT (quote y))) $
  stepsTrans (abs2Beta "rr" "xp"
    (Proofs.jTag (App (App Proofs.qMk (Var "xp")) (Var "rr")))
    (quote y) (quote x') (quoteClosed "rr" x')) $
  jTagCong (qMkBeta x' y)

||| stitching a right answer
dRightFin : (x, y' : Code) ->
  Steps (App (App (Proofs.jTag (Proofs.quote y')) Proofs.nTag)
             (App Proofs.rjT (Proofs.quote x)))
        (Proofs.jTag (Proofs.quote (QApp x y')))
dRightFin x y' =
  stepsTrans (jTagCase (quote y') nTag (App rjT (quote x))) $
  stepsTrans (abs2Beta "ll" "yp"
    (Proofs.jTag (App (App Proofs.qMk (Var "ll")) (Var "yp")))
    (quote x) (quote y') (quoteClosed "ll" y')) $
  jTagCong (qMkBeta x y')

mutual
  ||| the reader agrees with the engine on every stepping code
  simCorrectJ : (c, c' : Code) -> simStep c = Just c' ->
    Steps (App Proofs.simT (Proofs.quote c))
          (Proofs.jTag (Proofs.quote c'))
  simCorrectJ QS c' prf impossible
  simCorrectJ QK c' prf impossible
  simCorrectJ (QApp QS y) c' prf =
    let (y0 ** (meq, ceq)) = descRJust QS (simStep y) prf in
    rewrite ceq in
    stepsTrans (simEnter (QApp QS y)) $
    stepsTrans (appLs (appLs (rootReadL1S y))) $
    stepsTrans (nTagCase
      (App (App (App (quote (QApp QS y)) nTag) nTag)
           (App dTerm simT)) jWrap) $
    stepsTrans (dEnter QS y) $
    stepsTrans (appLs (appLs (simCorrectN QS Refl))) $
    stepsTrans (nTagCase
      (App (App (App simT (quote y)) nTag) (App rjT (quote QS)))
      (App ljT (quote y))) $
    stepsTrans (appLs (appLs (simCorrectJ y y0 meq))) $
    dRightFin QS y0
  simCorrectJ (QApp QK y) c' prf =
    let (y0 ** (meq, ceq)) = descRJust QK (simStep y) prf in
    rewrite ceq in
    stepsTrans (simEnter (QApp QK y)) $
    stepsTrans (appLs (appLs (rootReadL1K y))) $
    stepsTrans (nTagCase
```

```
      (App (App (App (quote (QApp QK y)) nTag) nTag)
           (App dTerm simT)) jWrap) $
    stepsTrans (dEnter QK y) $
    stepsTrans (appLs (appLs (simCorrectN QK Refl))) $
    stepsTrans (nTagCase
      (App (App (App simT (quote y)) nTag) (App rjT (quote QK)))
      (App ljT (quote y))) $
    stepsTrans (appLs (appLs (simCorrectJ y y0 meq))) $
    dRightFin QK y0
  simCorrectJ (QApp (QApp QK a) y) c' prf =
    rewrite sym (justInj prf) in
    stepsTrans (simEnter (QApp (QApp QK a) y)) $
    stepsTrans (appLs (appLs (rootReadK a y))) $
    stepsTrans (jTagCase (quote a)
      (App (App (App (quote (QApp (QApp QK a) y)) nTag) nTag)
           (App dTerm simT)) jWrap) $
    jWrapBeta (quote a)
  simCorrectJ (QApp xl@(QApp QS a) y) c' prf with (simStep a) proof eqa
    simCorrectJ (QApp xl@(QApp QS a) y) c' prf | Just a0 =
      rewrite sym (justInj prf) in
      stepsTrans (simEnter (QApp (QApp QS a) y)) $
      stepsTrans (appLs (appLs (rootReadL2S a y))) $
      stepsTrans (nTagCase
        (App (App (App (quote (QApp (QApp QS a) y)) nTag) nTag)
             (App dTerm simT)) jWrap) $
      stepsTrans (dEnter (QApp QS a) y) $
      stepsTrans (appLs (appLs
        (simCorrectJ xl (QApp QS a0) (cong (descR QS) eqa)))) $
      dLeftJ (QApp QS a0) y
        (App (App (App simT (quote y)) nTag) (App rjT (quote (QApp QS a))))
    simCorrectJ (QApp xl@(QApp QS a) y) c' prf | Nothing =
      let (y0 ** (meq, ceq)) = descRJust (QApp QS a) (simStep y) prf in
      rewrite ceq in
      stepsTrans (simEnter (QApp (QApp QS a) y)) $
      stepsTrans (appLs (appLs (rootReadL2S a y))) $
      stepsTrans (nTagCase
        (App (App (App (quote (QApp (QApp QS a) y)) nTag) nTag)
             (App dTerm simT)) jWrap) $
      stepsTrans (dEnter (QApp QS a) y) $
      stepsTrans (appLs (appLs
        (simCorrectN xl (cong (descR QS) eqa)))) $
      stepsTrans (nTagCase
        (App (App (App simT (quote y)) nTag)
             (App rjT (quote (QApp QS a))))
        (App ljT (quote y))) $
      stepsTrans (appLs (appLs (simCorrectJ y y0 meq))) $
      dRightFin (QApp QS a) y0
  simCorrectJ (QApp (QApp (QApp QS a) b) y) c' prf =
    rewrite sym (justInj prf) in
    stepsTrans (simEnter (QApp (QApp (QApp QS a) b) y)) $
    stepsTrans (appLs (appLs (rootReadS a b y))) $
    stepsTrans (jTagCase (quote (QApp (QApp a y) (QApp b y)))
      (App (App (App (quote (QApp (QApp (QApp QS a) b) y)) nTag) nTag)
           (App dTerm simT)) jWrap) $
    jWrapBeta (quote (QApp (QApp a y) (QApp b y)))
```

```
simCorrectJ (QApp xl@(QApp (QApp QK a) b) y) c' prf =
  rewrite sym (justInj prf) in
  stepsTrans (simEnter (QApp (QApp (QApp QK a) b) y)) $
  stepsTrans (appLs (appLs (rootReadL3K a b y))) $
  stepsTrans (nTagCase
    (App (App (App (quote (QApp (QApp (QApp QK a) b) y)) nTag) nTag)
         (App dTerm simT)) jWrap) $
  stepsTrans (dEnter (QApp (QApp QK a) b) y) $
  stepsTrans (appLs (appLs (simCorrectJ xl a Refl))) $
  dLeftJ a y
    (App (App (App simT (quote y)) nTag)
         (App rjT (quote (QApp (QApp QK a) b))))
simCorrectJ (QApp xl@(QApp (QApp (QApp w z) a) b) y) c' prf
  with (simStep xl) proof eqx
  simCorrectJ (QApp xl@(QApp (QApp (QApp w z) a) b) y) c' prf
    | Just x0 =
    rewrite sym (justInj prf) in
    stepsTrans (simEnter (QApp (QApp (QApp (QApp w z) a) b) y)) $
    stepsTrans (appLs (appLs (rootReadL3Deep w z a b y))) $
    stepsTrans (nTagCase
      (App (App (App
        (quote (QApp (QApp (QApp (QApp w z) a) b) y)) nTag) nTag)
        (App dTerm simT)) jWrap) $
    stepsTrans (dEnter (QApp (QApp (QApp w z) a) b) y) $
    stepsTrans (appLs (appLs (simCorrectJ xl x0 eqx))) $
    dLeftJ x0 y
      (App (App (App simT (quote y)) nTag)
           (App rjT (quote (QApp (QApp (QApp w z) a) b))))
  simCorrectJ (QApp xl@(QApp (QApp (QApp w z) a) b) y) c' prf
    | Nothing =
    let (y0 ** (meq, ceq)) =
      descRJust (QApp (QApp (QApp w z) a) b) (simStep y) prf in
    rewrite ceq in
    stepsTrans (simEnter (QApp (QApp (QApp (QApp w z) a) b) y)) $
    stepsTrans (appLs (appLs (rootReadL3Deep w z a b y))) $
    stepsTrans (nTagCase
      (App (App (App
        (quote (QApp (QApp (QApp (QApp w z) a) b) y)) nTag) nTag)
        (App dTerm simT)) jWrap) $
    stepsTrans (dEnter (QApp (QApp (QApp w z) a) b) y) $
    stepsTrans (appLs (appLs (simCorrectN xl eqx))) $
    stepsTrans (nTagCase
      (App (App (App simT (quote y)) nTag)
           (App rjT (quote (QApp (QApp (QApp w z) a) b))))
      (App ljT (quote y))) $
    stepsTrans (appLs (appLs (simCorrectJ y y0 meq))) $
    dRightFin (QApp (QApp (QApp w z) a) b) y0

||| the reader agrees with the engine on every silent code
simCorrectN : (c : Code) -> simStep c = Nothing ->
  Steps (App Proofs.simT (Proofs.quote c)) Proofs.nTag
simCorrectN QS prf =
  stepsTrans (simEnter QS) $
  stepsTrans (appLs (appLs rootReadAtomS)) $
  stepsTrans (nTagCase
```

```
    (App (App (App (quote QS) nTag) nTag) (App dTerm simT)) jWrap) $
  qSCase nTag nTag (App dTerm simT)
simCorrectN QK prf =
  stepsTrans (simEnter QK) $
  stepsTrans (appLs (appLs rootReadAtomK)) $
  stepsTrans (nTagCase
    (App (App (App (quote QK) nTag) nTag) (App dTerm simT)) jWrap) $
  qKCase nTag nTag (App dTerm simT)
simCorrectN (QApp QS y) prf =
  stepsTrans (simEnter (QApp QS y)) $
  stepsTrans (appLs (appLs (rootReadL1S y))) $
  stepsTrans (nTagCase
    (App (App (App (quote (QApp QS y)) nTag) nTag)
         (App dTerm simT)) jWrap) $
  stepsTrans (dEnter QS y) $
  stepsTrans (appLs (appLs (simCorrectN QS Refl))) $
  stepsTrans (nTagCase
    (App (App (App simT (quote y)) nTag) (App rjT (quote QS)))
    (App ljT (quote y))) $
  stepsTrans (appLs (appLs
    (simCorrectN y (descRNothing QS (simStep y) prf)))) $
  nTagCase nTag (App rjT (quote QS))
simCorrectN (QApp QK y) prf =
  stepsTrans (simEnter (QApp QK y)) $
  stepsTrans (appLs (appLs (rootReadL1K y))) $
  stepsTrans (nTagCase
    (App (App (App (quote (QApp QK y)) nTag) nTag)
         (App dTerm simT)) jWrap) $
  stepsTrans (dEnter QK y) $
  stepsTrans (appLs (appLs (simCorrectN QK Refl))) $
  stepsTrans (nTagCase
    (App (App (App simT (quote y)) nTag) (App rjT (quote QK)))
    (App ljT (quote y))) $
  stepsTrans (appLs (appLs
    (simCorrectN y (descRNothing QK (simStep y) prf)))) $
  nTagCase nTag (App rjT (quote QK))
simCorrectN (QApp (QApp QK a) y) prf impossible
simCorrectN (QApp xl@(QApp QS a) y) prf with (simStep a) proof eqa
  simCorrectN (QApp xl@(QApp QS a) y) prf | Just a0 impossible
  simCorrectN (QApp xl@(QApp QS a) y) prf | Nothing =
    stepsTrans (simEnter (QApp (QApp QS a) y)) $
    stepsTrans (appLs (appLs (rootReadL2S a y))) $
    stepsTrans (nTagCase
      (App (App (App (quote (QApp (QApp QS a) y)) nTag) nTag)
           (App dTerm simT)) jWrap) $
    stepsTrans (dEnter (QApp QS a) y) $
    stepsTrans (appLs (appLs
      (simCorrectN xl (cong (descR QS) eqa)))) $
    stepsTrans (nTagCase
      (App (App (App simT (quote y)) nTag)
           (App rjT (quote (QApp QS a))))
      (App ljT (quote y))) $
    stepsTrans (appLs (appLs
      (simCorrectN y
        (descRNothing (QApp QS a) (simStep y) prf)))) $
```

```
      nTagCase nTag (App rjT (quote (QApp QS a)))
simCorrectN (QApp (QApp (QApp QS a) b) y) prf impossible
simCorrectN (QApp (QApp (QApp QK a) b) y) prf impossible
simCorrectN (QApp xl@(QApp (QApp (QApp w z) a) b) y) prf
  with (simStep xl) proof eqx
  simCorrectN (QApp xl@(QApp (QApp (QApp w z) a) b) y) prf
    | Just x0 impossible
  simCorrectN (QApp xl@(QApp (QApp (QApp w z) a) b) y) prf
    | Nothing =
    stepsTrans (simEnter (QApp (QApp (QApp (QApp w z) a) b) y)) $
    stepsTrans (appLs (appLs (rootReadL3Deep w z a b y))) $
    stepsTrans (nTagCase
      (App (App (App
        (quote (QApp (QApp (QApp (QApp w z) a) b) y)) nTag) nTag)
        (App dTerm simT)) jWrap) $
    stepsTrans (dEnter (QApp (QApp (QApp w z) a) b) y) $
    stepsTrans (appLs (appLs (simCorrectN xl eqx))) $
    stepsTrans (nTagCase
      (App (App (App simT (quote y)) nTag)
           (App rjT (quote (QApp (QApp (QApp w z) a) b))))
      (App ljT (quote y))) $
    stepsTrans (appLs (appLs
      (simCorrectN y
        (descRNothing (QApp (QApp (QApp w z) a) b)
          (simStep y) prf)))) $
    nTagCase nTag (App rjT (quote (QApp (QApp (QApp w z) a) b)))
```

References

1. Project secret3, ‘A Quoted Alphabet’, ‘The Reader’s Engine’, ‘The First Eye’, ‘The Second Eye and the Assembler’, and ‘The Dispatcher’, working papers v0.1, August 11, 2026.
2. Project secret3, ‘Every Typed Song Ends’, working paper v0.1, August 11, 2026, for the finishing corollary.
3. A. M. Turing, ‘The p-function in lambda-K-conversion’, Journal of Symbolic Logic 2 (1937), for the fixed-point shape the knot follows.